# Autonomous Guided Vehicle Project

PMDS x DevNut Team

July 22, 2026

**Abstract**

This report documents the design and development of an Autonomous Guided Vehicle (AGV) system in a simulated warehouse environment. It describes the current controller, earlier controller iterations, the logging infrastructure and the web application used to inspect simulation results. Replace this text with a concise summary of the final system, its main design choices and the outcomes of the evaluation.

## Contents

# 1 Current Controller

This section describes the current AGV controller implemented for the Webots warehouse simulation. The controller integrates perception, localization, planning and low-level motion control to move each vehicle through the environment while accounting for configured goals and dynamic obstacles.

## 1.1 Architecture

Describe the controller as a sequence of cooperating components. Start from the information received from Webots and explain how sensor data is transformed into a state estimate, a route and finally motor commands. Refer to implementation details only when they clarify an engineering choice or a relevant limitation.

## 1.2 Planning and Control

Explain the high-level and low-level planning responsibilities, including how the controller selects waypoints, reacts to blocked paths and completes a goal. Use equations only when they make an algorithm or a measurable quantity clearer; otherwise, prefer a precise prose explanation.

## 1.3 Observed Behaviour

Summarize the behaviour observed during representative simulations. State what worked reliably, identify situations that required special handling and connect each limitation to its likely technical cause.

# 2 Previous Controller Iterations

The current controller described in Section 1 was preceded by two implementations, `OldController_1` and `OldController_2`, developed in sequence. Each iteration validated a specific part of the sense-think-act loop before the project committed to the modular architecture used today. This section records their design choices, what they demonstrated, and the concrete limitations that motivated the redesign, so that the evolution of the system remains traceable.

Beyond the control logic itself, each iteration also shaped the simulated environment it was tested against, from a small static arena to a warehouse world that eventually included obstacles capable of moving independently of the robot. As discussed below, that last change was not cosmetic: introducing moving obstacles altered what the perception and obstacle-avoidance code in `OldController_2` actually had to guarantee, and is one of the clearest examples of the environment driving a design decision rather than the other way around.

## 2.1 Initial Approach

Development started on a Gctronic **e-puck** robot placed in a small test arena, and only later moved to a **Pioneer 3-DX** robot inside the full-scale warehouse world. This substitution was a change of simulated hardware and environment scale, independent of the two controller logic iterations described below: `OldController_1` was written for the e-puck arena, while `OldController_2` was written for the Pioneer 3-DX warehouse world that the project still uses today. The two robot models differ substantially in size, which is reflected directly in the kinematic constants each controller hardcodes: `OldController_1` uses a wheel base of `0.052` m and a wheel radius of `0.02` m, while `OldController_2` uses `0.33` m and `0.0975` m respectively, matching the Pioneer 3-DX manual. The change in scale is also visible in the goal coordinates each controller targets: `OldController_1` drives toward a single hardcoded point near `(0.5, 0.5)`,

consistent with a small test arena, whereas `OldController_2` cycles through a list of warehouse-scale waypoints such as `(-29.3, 4)` and `(18.75, 2.25)`. Because the e-puck stage focused on validating the basic control and logging loop rather than warehouse-scale navigation, its lessons about obstacle-avoidance thresholds and acceleration behaviour do not translate directly to the Pioneer 3-DX scale, and had to be re-derived in `OldController_2`. The warehouse world itself grew in the same step, from a small empty arena to a layout with fixed shelving and, as described below, moving obstacles that `OldController_1` never had to contend with.

`OldController_1` implements a single-file, purely reactive controller whose purpose was to establish the minimal working pipeline: read the lidar and GPS, decide a wheel velocity, and persist a run history through the `RobotLog` service, tagged with `controller_version="old_controller"` so its runs remain distinguishable in the logs from later iterations. The only quantity it carries between simulation steps is the current wheel velocity read back from the motors; the controller keeps no explicit heading estimate. At every timestep it splits the 360-point lidar image, read by `read_lidar_image`, into three equal sectors (left, center, right) and computes the minimum range in each, and it derives a bearing toward the fixed goal directly from the raw GPS position with `get_angle`, rather than from any filtered state estimate. Sensor readings, wheel speeds and GPS coordinates are printed to the console roughly every half second, which was primarily a debugging aid for observing the controller live while the arena was small enough to watch directly. This console-first style of inspection was practical for a hand-sized arena and a single fixed goal, but it does not scale to a longer, multi-waypoint warehouse run in which relevant events are sparser in time and easy to miss in a scrolling terminal; this is part of the motivation for the more structured, periodically persisted logging that `OldController_2` adds, described later in this section.

Motion is selected through a fixed cascade of distance thresholds on the three sector minima: a full stop-and-reverse for a detected dead end, a hard turn for a close frontal obstacle, a gradual deceleration for a more distant frontal obstacle, and a small corrective turn when only a side sector is triggered; in the absence of any obstacle the robot ramps up to a cruise speed. The `acc_speed` helper applies linear acceleration and deceleration ramps toward the selected target speed so that velocity commands do not change discontinuously between steps, and `robot_to_wheel_velocity` converts the resulting linear and angular velocity into individual wheel angular velocities using the wheel base and wheel radius, saturating both wheels together so that the requested curvature is preserved even when the maximum wheel speed would otherwise be exceeded. This approach proved effective for validating that lidar readings, GPS, motor commands and the logging pipeline (obstacle-state tracking, idle-time accounting, event logging) worked together correctly on a simple, single-goal task. It does not scale to the warehouse scenario, however: the goal is a single point hardcoded in the main loop, so the controller has no notion of a route or of multiple waypoints; obstacle avoidance is purely threshold-based on the current sector minima, with no memory of previously seen obstacles and no deliberate path planning, so the robot can only react once it enters an obstacle's immediate vicinity rather than anticipate it; and the distance thresholds (for example `0.35` m for a frontal obstacle) were tuned for the e-puck's scale and would need to be re-derived for a Pioneer 3-DX operating in a much larger environment, which is exactly what `OldController_2` did.

`OldController_2` restructures the control logic into a `RobotController_v1` class built around explicit `State` and `Position` dataclasses, and introduces a list of sequential warehouse waypoints (`goal_positions`) that the robot advances through and loops over once the final goal is reached. It reuses the same wheel-velocity conversion and saturation pattern as `OldController_1` in `set_robot_velocity`, adapted to the Pioneer 3-DX's kinematic constants, and adds a camera device that is initialized and enabled but not yet consumed by the obstacle-avoidance logic, suggesting it was staged for a vision-based extension that was not completed in this iteration. Logging was extended accordingly: beyond the run history saved through `RobotLog`, the main loop periodically calls `save_to_database` roughly every five simulated seconds so that interme-

diate progress is persisted even if the simulation is interrupted.

The surrounding world was also extended with two `MovingWalls` instances, giving the controller dynamic obstacles to react to for the first time, in addition to the static warehouse layout. Each `MovingWalls` object locates a Webots node by its `DEF` name (`MOVING_WALL_1` and `MOVING_WALL_2`) through the controller robot's Supervisor interface, records that node's initial translation, and on every call to `move_wall` rewrites the corresponding coordinate of the node's `translation` field as the initial position plus a sinusoidal offset with a configured amplitude and frequency along a chosen axis. In the main loop this method is invoked unconditionally at the very start of every simulation step, ahead of state estimation, planning and logging: the first wall oscillates along the `y` axis with an amplitude of `3` m, the second along the `x` axis with an amplitude of `5` m, both driven at the same angular frequency of `0.1` rad/s. Because `getFromDef` and direct field mutation are Supervisor-only operations in Webots, running this code requires the `OldController_2` process itself to hold Supervisor privileges, so a single script ends up simultaneously responsible for driving the vehicle and for choreographing part of the environment it drives through, a coupling that `OldController_1` never needed since its arena was entirely static.

Critically, `RobotController_v1` has no dedicated code path for the moving walls: from the lidar's perspective a moving wall is indistinguishable from any other obstacle, and its motion only shows up indirectly, as occupied sector ranges that appear, disappear or shift between consecutive scans faster than a purely static obstacle's would. The two `MovingWalls` are therefore the concrete scenario in this iteration that exercises the sector-clustering logic's persistent-identity bookkeeping, described in the obstacle-avoidance discussion below, even though that logic itself does not treat static and dynamic obstacles any differently. In other words, the moving walls changed the requirements placed on the perception and avoidance code without requiring a single line written specifically for them: the obstacle-avoidance layer had to be correct for a world where sector boundaries can move on their own, and the walls were the mechanism by which that requirement was actually tested during development.

State estimation in `OldController_2` combines wheel odometry, integrated through `update_wheel_odometry` and `update_with_odometry`, with GPS correction in `fuse_with_gps`. In practice this fusion is configured with a GPS weight of `1.0`, so the corrected position is always the raw GPS reading and the odometry contribution is overwritten at every step rather than blended with it; the odometry-based prediction is computed but not actually used to stabilize the estimate. Heading and distance errors to the current goal are converted into linear and angular velocity commands by `calculate_velocity`, a proportional controller with gains `K_DISTANCE` and `K_HEADING` and a `cos(alpha)` term that slows the robot down when the heading error is large, with the resulting linear and angular velocities clamped to configured maxima and a small dead-zone applied to the angular command to avoid steady-state oscillation. Because this proportional scheme reacts only to the instantaneous heading and distance error, it carries no memory of previous corrections and no explicit model of the vehicle's dynamics, which is consistent with the controller's overall character as a direct, low-latency reaction to the current state estimate rather than a trajectory-tracking scheme.

Obstacle avoidance is the most developed part of this iteration. The lidar field of view is divided into 32 sectors, each classified as free or occupied, and consecutive occupied or free sectors are grouped into obstacle and free-space *clusters* that are assigned persistent identifiers across timesteps (`used_obstacle_ids`, `used_space_ids`, `previous_scan`), so that the same physical obstacle or gap keeps the same identity from one lidar scan to the next even as its exact sector boundaries shift. The two `MovingWalls` described above are the clearest source of this kind of shift: a purely static obstacle's sector boundaries change only as the robot itself moves, whereas a moving wall's boundaries also change on their own, independently of the vehicle's trajectory, so the persistent-identity mechanism has to hold up under a case the static warehouse layout alone would not exercise. The controller then locks onto a specific free-space cluster (`locked_space`)

while an obstacle blocks the direct path to the goal, which prevents it from oscillating between two competing free sectors on successive frames, and releases the lock once the obstacle is no longer relevant. When no free sector remains, the controller enters an escape mode that rotates in place until a free sector reappears. This sector-clustering strategy proved robust enough that, as discussed below, it was carried forward almost unchanged into the current controller.

The implementation also exposes concrete limitations. The persistent-identity logic that tracks clusters across frames is intricate and stateful, and the code raises an exception if a sector is left without an assigned identifier, which is a fragile failure mode for a long-running simulation. The file still contains earlier, commented-out obstacle-avoidance attempts (`old_obstacle_avoidance`, `min_distances`), which indicates that the sector-based strategy was reached iteratively rather than designed upfront. Goal waypoints and robot-specific parameters remain hardcoded in the controller script instead of being externalized to a configuration file. The Supervisor privileges required to animate the `MovingWalls` are a further limitation: granting a per-robot controller direct write access to arbitrary scene nodes is difficult to justify once several AGVs share the same warehouse, since any one controller process could in principle reposition world geometry that every other vehicle relies on for localization and obstacle avoidance, and `OldController_2` does not scope or guard that access in any way.

### 2.2 Lessons Learned

Comparing the two iterations with the current controller (Section 1) shows a clear line of continuity rather than a full rewrite. The sector-clustering obstacle-avoidance algorithm introduced in `OldController_2` is, in its essential structure, the same algorithm implemented today by the `SectorPlanner` low-level planner: the same notion of dividing the lidar field of view into sectors, assigning persistent obstacle and free-space identifiers across frames, locking onto a chosen free sector, and rotating to escape a dead end. This is direct evidence that the local, reactive avoidance layer designed and tuned in `OldController_2` was robust enough to be promoted into production, once decomposed behind a dedicated planner interface and driven by configuration objects instead of hardcoded constants. The `MovingWalls` introduced in `OldController_2` were, in this sense, an early stress test for that design: because they could enter or leave a sector independently of the robot's own trajectory, they exercised the persistent-identity bookkeeping in a way a purely static layout would not, and the fact that the same clustering approach was carried forward into the `SectorPlanner` suggests it held up under that test rather than being reworked to cope with dynamic obstacles later. The lower-level wheel-velocity conversion pattern first written in `OldController_1` and reused in `OldController_2` survived the transition as well, which suggests that the basic differential-drive actuation model was correct from the earliest iteration and needed no substantial rework, only re-parameterization for each robot's physical dimensions.

Other aspects of the design changed more substantially. Route information moved out of the controller code entirely: instead of the hardcoded `goal_positions` list, waypoints and per-robot routes are now declared in `goals.config.json` as named locations, which decouples warehouse layout changes from controller logic and lets new pickup or dropoff points be added without touching Python code. The monolithic `RobotController_v1` class was split into dedicated perception, localization, planning and hardware modules, and a high-level planner was introduced above the reactive sector layer so the vehicle now follows a globally planned route rather than only locally reactive corrections. Hardware access was also wrapped behind a dedicated hardware-interface layer instead of calling `robot.getDevice` directly from control logic, improving testability and making the sensor and motor interfaces explicit; the camera device that `OldController_2` enabled but never used is, in this later architecture, finally exercised by a dedicated perception module.

The GPS/odometry fusion limitation identified in `OldController_2` was also addressed directly rather than carried forward silently: the current localization module generalizes the fixed

`gps_weight = 1.0` into a configurable weighted fusion, adds a disagreement threshold that snaps the state estimate to GPS only when odometry has drifted too far, and additionally fuses a heading estimate from the inertial unit, which the old controllers did not use at all. Taken together, these observations show that the old controllers were most useful not as discarded prototypes but as a way to isolate which parts of the design were already sound and which parts still needed rework: the sector-based reactive avoidance layer and the differential-drive actuation model proved robust enough to keep, while state estimation, goal configuration and, most importantly, coordination between multiple AGVs sharing the same warehouse still required further work before the system could operate reliably at full scale.

# 3 Logging Infrastructure

The logging subsystem provides a persistent record of simulations, events and telemetry generated by the AGVs. Its role is to make the execution of a simulation inspectable after the fact and to provide the data consumed by monitoring tools. The pipeline is built around a client-server architecture: each robot controller collects events in memory and periodically flushes them to a standalone HTTP service backed by MySQL.

## 3.1 Service Architecture

The logging pipeline follows a client-server model. Each Webots robot controller process runs an instance of the `RobotLog` class in `logging/logger/robot_log.py`, which acts as an in-memory event collector and HTTP client. A separate process hosts the logging server defined in `logging/logging_server.py`, which exposes a set of HTTP endpoints split between write and read responsibilities, all served by Python's `ThreadingHTTPServer`. Using a thread-per-request model allows the server to handle concurrent flush operations from the two robots deployed in the warehouse world and read requests from the monitoring web application without blocking any client.

The write-side and liveness endpoints serve distinct roles in the simulation lifecycle:

- `POST /start` accepts a `unit_id` string, creates a new row in the `Simulations` table, and returns the assigned `sim_id` to the client. This identifier anchors every subsequent event and telemetry record for the run.
- `POST /save` accepts a batch payload containing updated summary fields, a list of events and a list of telemetry records. The server validates the payload, opens a database transaction, and atomically writes all three components to MySQL. An HTTP 200 response indicates success; a 400 or 500 response signals validation or infrastructure failure, respectively.
- `GET /health` returns `{"status": "success"}` and serves as a liveness probe for the caller.

The read side of the API serves the monitoring web application described in section 4. Five `GET` endpoints expose committed simulation data: `/simulation-options` lists the available unit identifiers together with their simulation identifiers; `/simulations`, `/events` and `/event-telemetry` return rows from the corresponding database tables; and `/database-snapshot` returns all three collections in a single consistent response. Each filtering endpoint accepts optional `unit_id` and `sim_id` query parameters that restrict the result to one robot or one run. The design, validation and concurrency handling of these endpoints are detailed in section 3.6.

The server listens on `127.0.0.1:8080` by default. Command-line arguments `-host` and `-port` override the listener address, while the `LOGGING_SERVER_URL` environment variable controls the URL used by the robot controllers. This layered configuration supports local development, where the server and both controllers run on the same machine, and any future extension to a distributed setup.

## 3.2 Data Model

The server persists simulation data in three MySQL tables created by the queries in `Database_Structure.sql`. Each table addresses a distinct level of granularity: run-level aggregates, discrete event records, and per-event controller state snapshots. Figure 1 summarises the three tables, their columns and the relationships between them; the remainder of this subsection describes each table in turn.

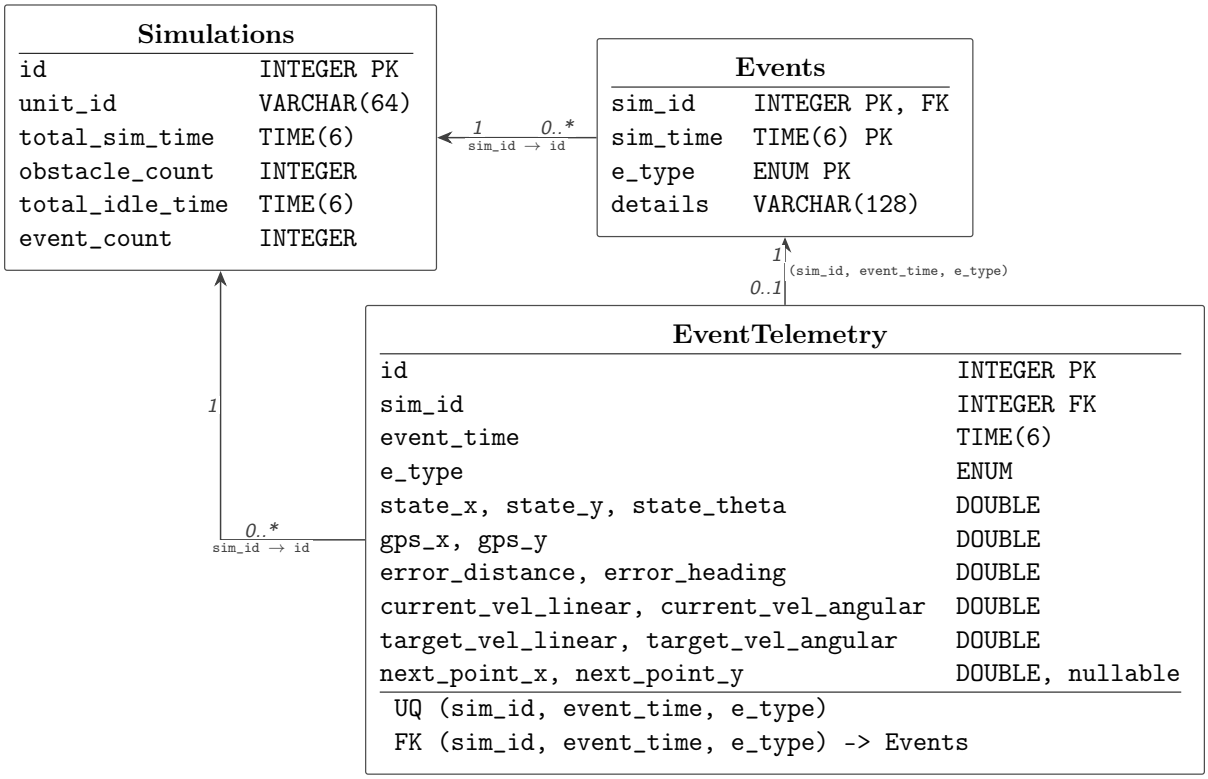


Figure 1: UML-style schema of the logging database. Each arrow points from a foreign key in the child table to the key that it references in the parent table. Multiplicities describe the number of child rows associated with one parent row. `PK`, `FK` and `UQ` denote primary-key, foreign-key and unique constraints, respectively.

**Simulations.** The `Simulations` table holds one row per completed run. Its auto-increment `id` serves as the primary key and is the foreign key target of the event-related tables. Each row stores the robot's `unit_id` as a variable-length string and four aggregate metrics: `total_sim_time`, `obstacle_count`, `total_idle_time` and `event_count`. Temporal columns use the MySQL `TIME(6)` type with microsecond precision, preserving sub-second resolution for runs that span tens of minutes.

**Events.** The `Events` table records discrete state transitions and notable occurrences. Its composite primary key `(sim_id, sim_time, e_type)` ensures that a simulation cannot contain two events of the same type at the same instant. The `e_type` column is modeled as a MySQL `ENUM` restricted to eight values:

- `START` — logged when the controller confirms a new simulation record.
- `STOP` — logged during controller shutdown.
- `IDLE_START` and `IDLE_END` — mark transitions when the robot's linear velocity crosses a near-zero threshold.

- `OBSTACLE_ENCOUNTER` and `OBSTACLE_CLEARED` — reflect the presence or absence of dynamic obstacles detected by the low-level safety module.
- `REACHED_TARGET` — recorded when the controller's distance to the current goal falls below the configured threshold.
- `UNEXPECTED_BEHAVIOR` — an open-ended category for anomalous conditions observed by the controller.

Each event carries a `details` column of up to 128 characters. For obstacle events the details encode the GPS coordinates at which the obstacle was detected or cleared; for idle transitions they record the linear and angular velocity that triggered the state change; for target events they include the goal index and coordinates. This compact textual context makes the event log self-describing when inspected without joining against telemetry.

**EventTelemetry.** The `EventTelemetry` table extends each event with a complete snapshot of the controller's internal state. Its 14 numeric columns capture the estimated pose (`state_x`, `state_y`, `state_theta`), the GPS reading (`gps_x`, `gps_y`), the control errors computed from the current pose to the goal (`error_distance`, `error_heading`), the current linear and angular velocity (`current_vel_linear`, `current_vel_angular`), the command velocities output by the control module (`target_vel_linear`, `target_vel_angular`), and the coordinates of the next waypoint on the low-level path (`next_point_x`, `next_point_y`). The waypoint fields are nullable because the planner may produce an empty path under certain conditions; all other fields are non-nullable `DOUBLE` columns.

A `UNIQUE` constraint on `(sim_id, event_time, e_type)` mirrors the `Events` composite primary key, allowing each event to have at most one telemetry row. The relationship is therefore one-to-zero-or-one from `Events` to `EventTelemetry`: a telemetry row must reference an event, while the schema does not require every event to have telemetry. A foreign key from `(sim_id, event_time, e_type)` to the corresponding columns in `Events` enforces this referential integrity.

### 3.3 Data Flow and Controller Integration

The logging lifecycle is tightly coupled to the controller's execution in `AGVSimulation.run()` (`DefaultController.py:74`). During initialisation, the constructor instantiates a `RobotLog` with the configured server URL and the robot's `unit_id`, then calls `logger.start()`. This method sends a `POST /start` request and, upon success, stores the returned `sim_id` and logs a `START` event on the client side. If the server does not respond successfully within ten attempts, spaced one second apart, the method returns `False`. The controller responds by calling `_refuse_to_run()`, which stops the motors and leaves the robot immobile for the remainder of the simulation. This design ensures that no robot operates without a persistent audit trail.

Once the simulation is running, the controller's main loop executes at the Webots step rate. On every cycle, three logging operations take place:

1. `capture_telemetry()` (`robot_log.py:233`) stores the latest state estimate, GPS reading, control command and low-level waypoint in the `latest_telemetry` dictionary. This snapshot is held in memory and attached to the next event that is logged. The method also receives the current simulation time and forwards the same data to the realtime mirror described in section 3.5, which writes a live snapshot of the robot's state to disk for the monitoring dashboard.
2. `update_obstacle_state()` (`robot_log.py:297`) compares the current obstacle presence flag from the low-level planner's safety status with the previous state. A transition from clear to obstructed produces an `OBSTACLE_ENCOUNTER` event; the reverse transition produces `OBSTACLE_CLEARED`. The obstacle count is incremented once per encounter, not per cycle, giving a meaningful count of distinct obstruction episodes.

3. `update_idle_state()` (`robot_log.py:311`) detects transitions based on whether the robot's linear velocity magnitude falls below the threshold `idle_speed_threshold` (default $10^{-3}$). It accumulates the total idle time and logs `IDLE_START` and `IDLE_END` events at each transition.

Additionally, `log_target_reached()` is called from the controller's goal-handling logic (`DefaultController.` when the distance error to the current target drops below `goal_reached_thresh`. This produces a `REACHED_TARGET` event that includes the goal index and coordinates.

Rather than sending each event individually, the `RobotLog` accumulates events and their matching telemetry records in Python lists and flushes them in periodic batches. The flush interval is controlled by `LogConfig.log_interval` in `config.py`, set to 5 seconds by default. The decision to batch events rather than stream them avoids saturating the HTTP server with a request per simulation step (which runs at hundreds of hertz) and reduces the number of database transactions.

The batch flush procedure in `RobotLog.flush()` (`robot_log.py:356`) uses a two-queue design. The `failed_events` and `failed_event_telemetry` lists hold records that could not be persisted in a previous attempt. On each flush, the failed queue is retried first by calling `_send_batch()`. If the retry succeeds, the failed queues are cleared and the current buffer is sent. If the retry fails, the current buffer is appended to the failed queues, preserving the chronological order of events across multiple failed attempts. Only after a successful round-trip are events removed from both the main and failed queues.

When the simulation terminates, the `finally` block of `run()` (`DefaultController.py:143`) calls `logger.stop()`, which logs a `STOP` event and enters a retry loop for flushing all remaining data, including the failed queue. If the server remains unreachable after ten attempts, the method prints the number of unsaved records and returns `False`, accepting controlled data loss at a well-defined shutdown boundary. A nested `finally` block then calls `logger.clear_realtime()` (`robot_log.py:160`) to truncate the unit's live snapshot file, so the monitoring dashboard does not continue to display a stale pose for a robot that is no longer running.

### 3.4 Validation and Transactional Integrity

The server enforces payload correctness before any data reaches the database. The validation function `validate_save_payload()` (`logging_server.py:120`) performs a comprehensive check of every field in the request body. Numeric fields must be finite; integer fields must be of type `int` and nonnegative; string fields must respect length limits; the `e_type` of every event and telemetry record must belong to the `ALLOWED_EVENT_TYPES` frozenset; and the `sim_id` embedded in each sub-object must match the top-level `sim_id`. The validator also constructs sets of event identities, defined as tuples of `(sim_id, time, e_type)`, and verifies that the identities from the `events` and `event_telemetry` arrays match exactly and contain no duplicates. This cross-consistency check catches misalignments between the two arrays before the database transaction begins.

When validation fails, the function raises `PayloadValidationError` with a message identifying the offending field. The HTTP handler catches this exception and returns a 400 response, allowing the controller to distinguish between malformed data (which cannot be fixed by retrying) and infrastructure failures (which may be transient).

The `PersistenceService.save()` method (`logging_server.py:465`) writes accepted data inside a single database transaction. It first acquires a row-level lock on the target `Simulations` row with `SELECT ... FOR UPDATE` and verifies that the stored `unit_id` matches the requesting unit. This ownership check prevents a controller from accidentally or maliciously overwriting another robot's simulation data. The method then updates the summary columns and inserts events and telemetry rows using `executemany()` with `ON DUPLICATE KEY UPDATE` clauses. The idempotent insert makes the operation safe to retry: if a previous flush succeeded but the response

was lost, re-sending the same batch will not create duplicate rows.

If any database operation raises an exception during the transaction, the entire batch is rolled back and a `PersistenceError` is raised, which the HTTP handler maps to status 500. The `save()` method treats `PayloadValidationError` specially: it rolls back the transaction but does not wrap the exception, allowing the handler to return status 400. This separation of error classes keeps the controller informed about the nature of each failure.

Concurrency is managed at two levels. The `PersistenceService` uses a `threading.Lock` to serialise every `save()` and read method, ensuring that transactions from concurrent flush operations by the two robots and concurrent read requests do not interleave. The MySQL connection is not held open between flushes; instead, `_get_connection()` (`logging_server.py:226`) checks the liveness of any cached connection with `ping(reconnect=True)` and creates a new connection when needed. This lazy-reconnect strategy avoids the complexity of connection pooling for a workload where writes arrive no more than every five seconds and reads are short `SELECT` statements.

### 3.5 Realtime Telemetry Mirror

The batched flush design described in section 3.3 makes committed data available to the database only once every few seconds, and always at least one flush interval behind the controller's current state. For the monitoring web application to render a live view of the robots in motion, it needs a more recent signal than the committed record can provide. The `RobotLog` class therefore maintains a second, independent output channel: a single-record JSONL file that always reflects the robot's latest controller cycle.

Each time `capture_telemetry()` is called, it delegates to `_write_realtime_snapshot()` (`robot_log.py:117`), which serialises the current simulation time, estimated pose, GPS reading, control errors, current and target velocities, goal position and next waypoint into a compact JSON object. The payload is written to a file named `robot_controller_runs_{unit_id}_realtime.jsonl` in the `logging/logs` directory, where `unit_id` is the robot's numeric identifier. Rather than appending, the writer replaces the file contents atomically: it writes to a temporary file in the same directory and then calls `os.replace()` to swap it into place. This is the same temporary-file-and-rename pattern used by the server-side JSONL mirror (section 3.7), and it guarantees that a reader never observes a partially written record.

Writing on every controller cycle would exceed the refresh rate the dashboard can usefully display and would add avoidable disk traffic. The writer therefore throttles updates with `REALTIME_WRITE_INTERVAL` (`robot_log.py:11`), set to `0.05` s by default, so that the snapshot is refreshed at most twenty times per second. The throttle compares successive simulation timestamps rather than wall-clock time, keeping the update rate tied to simulation progress. A non-numeric `unit_id` disables the mirror entirely, since the filename scheme assumes a numeric identifier; the writer prints a diagnostic and returns `False` in that case.

The file is intended to hold exactly one line, representing the most recent state. When the controller shuts down, the nested `finally` block in `AGVSimulation.run()` calls `clear_realtime()` (`robot_log.py:160`), which overwrites the file with empty content using the same atomic-replace primitive, so that the dashboard does not keep displaying a robot that has stopped. Because the realtime mirror is written by the controller process and read from disk, it is independent of both the HTTP server and the database: a database outage does not interrupt the live view, and the live view does not compete for database locks. The trade-off is that the mirror carries only the current state, not a history. Once a snapshot is overwritten the previous state is lost, and the committed event and telemetry tables remain the authoritative record of past behaviour.

### 3.6 Server-Side Read API

The logging server exposes five GET endpoints that let clients retrieve committed data without direct access to MySQL. They serve the monitoring web application's historical and tabular views, complementing the live realtime mirror described in section 3.5. All read endpoints share the same ThreadingHTTPServer and PersistenceService instance as the write path, and they are dispatched by the do_GET handler (logging_server.py:645).

**Endpoints and filtering.** GET /simulation-options takes no parameters and returns the set of unit_id values present in the Simulations table, each paired with the list of its simulation identifiers ordered most-recent-last. It lets the dashboard populate unit and run selectors before issuing more specific queries. GET /simulations, GET /events and GET /event-telemetry return rows from the corresponding tables, while GET /database-snapshot returns all three collections in a single response; the last is the endpoint the web application uses for its tabular view, since one request yields a self-consistent picture of a run. Every filtering endpoint accepts optional unit_id and sim_id query parameters, validated by validate_read_query() (logging_server.py:76), which rejects any other parameter name, requires each filter to appear exactly once, and normalises both values to numeric strings or positive integers. validate_empty_query() (logging_server.py:104) enforces that /simulation-options and /health receive no query string. As on the write path, a PayloadValidationError maps to HTTP 400 and a PersistenceError to 500.

**Result limiting and ordering.** Each read query is capped at DEFAULT_READ_LIMIT (logging_server.py:17) set to 200 rows. The underlying SELECT statements order by the primary identifier in descending order and apply a LIMIT clause, after which the result list is reversed in Python. This combination returns the most recent 200 rows in chronological order, bounding the response size for endpoints that could otherwise return the full history of every run. The same limit applies to the individual endpoints and to /database-snapshot.

**Consistency and concurrency.** The /database-snapshot endpoint is served by PersistenceService.read (logging_server.py:413), which opens a single transaction with start_transaction(consistent_snapshot=T readonly=True) before issuing the three SELECT queries. Because all three reads execute inside one consistent read snapshot, the simulations, events and telemetry arrays cannot diverge even if a concurrent flush commits new data between the individual queries. The other read endpoints issue a single query each and therefore need no snapshot transaction. Like the write path, every read method acquires the PersistenceService instance lock and reuses the cached MySQL connection through _get_connection(). Reads never commit: the finally block of each method calls _rollback() to end the read transaction and release the snapshot, leaving the connection ready for the next request.

**Response caching and serialisation.** Read responses carry a Cache-Control: no-store header, set by the no_store flag in _send_json() (logging_server.py:722), which instructs the browser and any intermediate proxy not to cache the payload. This prevents the dashboard from displaying stale data when a user refreshes a view between simulation updates. The same method serialises responses with PersistenceService._json_default() (logging_server.py:632), which converts the timedelta, datetime and time objects returned by the MySQL connector into strings and Decimal values into floats, so that temporal columns such as total_sim_time and sim_time survive the JSON round-trip without manual conversion in the caller.

### 3.7 JSONL Mirror and Dashboard Integration

After every successful database commit, `save()` invokes `_best_effort_rebuild()` (`logging_server.py:560`), which in turn calls `rebuild_jsonl()` (`logging_server.py:566`) to regenerate three flat files in the `logging/logs` directory: `simulations.jsonl`, `events.jsonl` and `event_telemetry.jsonl`. Each file contains one JSON object per line, with keys derived from the MySQL column names. The rebuild issues its `SELECT` queries on the cached connection after commit, reading from a consistent snapshot of the committed state, so the files faithfully represent the data just written. The call is wrapped in a best-effort guard: if file regeneration raises, the server logs a warning but leaves the already-committed database rows intact.

The files are written atomically using a temporary-file-and-rename pattern (`_write_jsonl_atomic()`, `logging_server.py:610`). Data is written to a temporary file in the same directory, flushed to disk with `os.fsync()`, and then moved to the final destination with `os.replace()`, which is atomic on the same filesystem. If any exception interrupts the process, the temporary file is removed. A reader therefore always sees a complete, self-consistent snapshot rather than partial content.

This mirror serves primarily as a durable, human-readable archive of every simulation run. A developer can inspect the latest simulation with `tail simulations.jsonl`, search for specific event types across all runs with `grep`, or import the data into a Python notebook for statistical analysis, without navigating the MySQL interface. The monitoring web application does not read these files directly: its live view consumes the client-side realtime mirror of section 3.5, while its committed-data views go through the read API of section 3.6. The archive and the read API nevertheless present the same committed state, since both are derived from the database; the archive optimises for ad-hoc human inspection, the API for structured client consumption.

Together the two read channels cover the dashboard's two temporal modes. While a simulation is in progress, the realtime mirror offers a low-latency view of each robot's current pose and velocity, read from disk without involving the logging server or the database. Once data has been flushed and committed, the read API provides the consistent, filtered snapshots used by the historical and tabular views. This separation keeps the live display responsive and the committed display accurate, at the cost of a brief window in which the live view may be ahead of the last committed flush.

### 3.8 Reliability Considerations

The logging system adopts a conservative approach to reliability that prioritises data completeness over availability during startup while tolerating transient failures during steady-state operation.

The most consequential decision is the controller's refusal to move if the logging service cannot create a simulation record at startup, enforced in `RobotLog.start()` (`robot_log.py:211`). Ten consecutive HTTP failures are treated as a terminal condition; the robot remains immobile for the entire simulation. This stop-the-world posture is chosen because a missing simulation record would render all subsequent events unanchored. The `Events` and `EventTelemetry` tables carry foreign keys referencing `Simulations.id`, and without a valid `sim_id` no event data can be committed to the database. Buffering events indefinitely in hope of a late recovery would risk unbounded memory growth and opaque data loss, making post-hoc debugging harder.

During normal operation the batch-flush design trades some timeliness for robustness. Events spend up to five seconds in controller memory before being sent to the server. A crash of the controller process or Webots within a flush interval loses at most the events accumulated since the last successful flush. The failed-events queue absorbs transient server unavailability: if the server is down when a batch is attempted, the records are preserved and retried on the next interval without interrupting the simulation loop. This decoupling ensures that a brief database outage does not halt the robots, at the cost of potentially losing the in-memory batch if the

controller itself terminates before the next successful flush.

At shutdown, the `RobotLog.stop()` method makes up to ten additional flush attempts. Data that remains unsaved after these attempts is discarded, and a console message reports the count. This explicit best-effort-accept-loss boundary avoids blocking the application exit indefinitely while making the extent of any data loss visible to the operator.

The MySQL container is provisioned via Docker, as documented in the project `README.md`. The database connection parameters (host, port, user, password, database name) are read from environment variables with defaults that match the Docker setup, allowing the server to be reconfigured without code changes. The container runs on the standard MySQL port 3306 and stores its data in a Docker volume that persists across container restarts.

### 3.9 Use in Evaluation

The structured event log is the primary source of quantitative evidence for evaluating controller behaviour. The `EventTelemetry` table records the robot's full estimated state at every logged moment, enabling offline reconstruction of trajectories, analysis of convergence toward goals, and comparison of planning decisions across different controller configurations or environment layouts.

The summary metrics aggregated per simulation run (`total_sim_time`, `total_idle_time`, `obstacle_count`, `event_count`) support direct quantitative comparison between runs. For example, total idle time can serve as a proxy for navigation efficiency: a robot that waits for obstacles to clear contributes more idle time than one that finds alternative routes. The obstacle count reflects the frequency of dynamic interference, which varies with the movement patterns of the supervisor-controlled humans and forklifts in the environment. Because each simulation row is identified by its `unit_id`, results from the two robots operating concurrently can be compared side by side, controlling for the same world conditions.

The `details` column of the `Events` table provides a lightweight narrative of each run. Searching for `OBSTACLE_ENCOUNTER` events reveals the locations and frequencies of obstruction episodes; inspecting the GPS coordinates in the details string can identify bottlenecks in the warehouse layout where dynamic obstacles cluster. `UNEXPECTED_BEHAVIOR` events capture anomalous conditions that the controller explicitly flags, making them immediately searchable without scanning the full telemetry.

The JSONL mirror enables ad-hoc analysis with standard command-line tools. A developer can extract all events of a given type with `grep`, compute aggregate statistics from the telemetry file with a short script, or load the data into a Jupyter notebook for more involved analysis. The web application described in section 4 renders the monitoring dashboard from the read API and the realtime mirror described above, while the JSONL archive remains available for direct programmatic inspection. Both interfaces are derived from the same committed database state, so ad-hoc analysis and visual oversight stay consistent with one another.

## 4 Web Application

The web application provides an operational view of the AGV simulations. It combines the data exposed by the logging pipeline with a visual representation of the warehouse, allowing users to inspect vehicle status, configured goals and the progression of a run.

### 4.1 Interface Design

Describe the principal views of the application and the information each view presents. Explain how the interface supports the workflow of monitoring a live simulation and reviewing a completed one.

## 4.2 Data Integration

Explain how the web application obtains and transforms simulation data before rendering it. Mention refresh behaviour, data formats and any consistency constraints that affect what users see.

## 4.3 Practical Value and Limitations

Summarize the value of the dashboard for development and demonstrations. Then state the current limitations, such as dependence on the logging service or any incomplete visualizations, and identify the most useful future improvements.